TRUSTQA

PRODUCT REQUIREMENTS DOCUMENT

A Multi-Document RAG System with Source-Verified Answers for Scanned and Tabular Documents

Project purpose

TrustQA is a document question-answering application that allows a user to upload several PDFs and ask questions across them. It is designed for machine-readable PDFs, scanned/image-based PDFs and documents containing tables. The system retrieves relevant passages, generates an evidence-grounded answer, checks its support and displays the source document, PDF page, supporting chunk and verification status.

The defining product principle is not simply to generate an answer, but to make the answer traceable to retrieved evidence and the original document.

PRD item	Definition
Project name	TrustQA
Project type	Four-member undergraduate software / NLP project
Institution	RV University – Department of Computer Science and Engineering
Primary users	Students, researchers and document reviewers working with PDF collections
Target prototype	Python pipeline in Google Colab; Streamlit demonstration interface
Version / date	1.0 | 31 October 2026
Status	Proposed requirements; implementation and evaluation pending

Source basis: the TrustQA nine-page research report and the team's agreed implementation plan (15 initial PDFs, 50 manually checked questions, four modules and baseline-versus-TrustQA evaluation). Requirements below are proposed implementation details, not claims that the system is already built.

1.1 Problem statement

Real-world document collections are heterogeneous. Important facts may be stored in scanned pages, tables, forms, reports or several related documents. OCR errors can affect later retrieval and generation. Flattening tables can destroy row-column relationships. Semantic retrieval may miss exact values, names and dates or fail to gather evidence distributed across documents. Even when relevant evidence is retrieved, a language model may generate unsupported or contradictory claims. A citation generated by a model does not, by itself, prove that the cited page supports the answer.

1.2 Product vision and goals

- Support question answering over multiple uploaded PDFs in one collection.
- Process digital PDFs and scanned PDFs while preserving page-level provenance.
- Preserve table headers, rows and values for table-aware retrieval and numerical questions.
- Retrieve relevant evidence across documents and return original metadata.
- Generate answers only from retrieved evidence and abstain when evidence is insufficient.
- Check citation identity/page mapping separately from whether evidence supports the claim.
- Expose warnings for unsupported claims, OCR uncertainty, numerical mismatches and conflicting sources.
- Measure whether the full TrustQA pipeline improves reliability compared with basic RAG.

1.3 Intended users and use cases

User	Need / use case
Student or researcher	Ask questions across research papers, reports or manuals and inspect supporting pages.
Analyst / reviewer	Find values and compare information across reports while retaining source context.
Project evaluator	Inspect handling of scanned content, tables, conflicts and unsupported questions.

1.4 Scope boundaries

In scope for prototype	Out of scope / optional
PDF upload; digital text extraction; page-level OCR fallback; table extraction; metadata-rich chunks; semantic retrieval; grounded answer; citation checks; evidence status; Streamlit demo; evaluation.	Production-scale deployment, user accounts/permissions, automatic web research, guaranteed OCR correctness, domain-specific fine-tuning, advanced knowledge graphs, complex adaptive RAG and costly reranking.

Scope rule: deliver a reliable end-to-end baseline first. Add hybrid retrieval, reranking or advanced reasoning only after the core workflow and data contracts pass tests.

2.1 End-to-end workflow

Upload PDFs → extract page text / OCR → extract tables → normalise and chunk → create embeddings and index → retrieve evidence → generate answer → verify claims and citations → display answer, sources, status and warnings.

2.2 Functional requirements

ID	Requirement	Priority
FR-01	Upload multiple PDFs and assign a stable document_id to each.	Must
FR-02	Extract digital PDF text page by page; retain original filename and 1-indexed PDF page number.	Must
FR-03	Detect pages with little/no extractable text and run Tesseract OCR fallback; record extraction_method and ocr_used.	Must
FR-04	Extract tables with pdfplumber; preserve table_id, headers, rows, cells, units and source page.	Must
FR-05	Create unique chunk IDs and retain document/page/type metadata throughout the pipeline.	Must
FR-06	Create embeddings with Sentence Transformers and index with FAISS; return top-k original records.	Must
FR-07	Accept a natural-language question and retrieve evidence from one or more uploaded documents.	Must
FR-08	Generate an answer grounded only in retrieved passages; abstain when evidence is inadequate.	Must
FR-09	Validate citation chunk IDs and resolve document/page from trusted metadata, not LLM-generated fields.	Must
FR-10	Assess whether cited passages support answer claims; detect unsupported or conflicting claims.	Must
FR-11	Check numerical values, units and periods against source evidence; use deterministic calculations where possible.	Should
FR-12	Return answer, citations, status and warnings in the common output schema.	Must
FR-13	Provide Streamlit upload, question and evidence/status interface.	Should
FR-14	Compare basic RAG and TrustQA using a fixed evaluation set and documented settings.	Must
FR-15	Optionally combine BM25 keyword retrieval with semantic retrieval after baseline testing.	Could

2.3 Verification status rules

Status	Meaning / behaviour
Verified	All material factual claims are supported by supplied evidence and cited chunk IDs resolve to the correct document and page.
Partially Supported	Some claims are supported, but others are incomplete, uncertain or conflicting. Explain the limitation.
Insufficient Evidence	The uploaded collection does not support a reliable answer. Abstain rather than guessing.

A citation ID check validates provenance only. A separate evidence-support check is required to assess whether the cited passage actually supports the claim.

3.1 Architecture components

Component	Proposed implementation	Output
PDF ingestion	Python file handling; stable document registry	Uploaded document list
Text extraction	PyMuPDF, page by page	Page-level text records
OCR fallback	Tesseract via pytesseract on flagged pages	OCR text with page provenance
Table processing	pdfplumber; Camelot optional for suitable digital PDFs	Structured headers / rows / cells
Chunking	Configurable size and overlap; table-aware row representation	Canonical chunk records
Embedding / index	Sentence Transformers + FAISS CPU	Vector index and aligned metadata
Optional hybrid	rank-bm25 + rank fusion	Combined ranked evidence
Generation	Configurable LLM API/local model wrapper	Grounded draft answer and proposed chunk citations
Verification	Metadata checks, claim support and numeric checks	Validated citations, status and warnings
Interface	Streamlit	Upload/question/result experience

3.2 Canonical chunk data contract

```
{
"document_id": "DOC01",
"document_name": "annual_report.pdf",
"page_number": 4,
"chunk_id": "DOC01_P4_C1",
"text": "The company reported revenue of Rs. 12 crore in 2025.",
"content_type": "text",
"table_data": null
}
```

Required fields: document_id, document_name, page_number, chunk_id, text, content_type and table_data. For table chunks, content_type is “table” and table_data preserves table_id, headers and rows. The text field is a row-aware search representation; it must not replace the structured table. Use 1-indexed PDF page positions consistently. Retain extraction_method, ocr_used and section when available.

3.3 Module interfaces

Module	Function contract
1. Document processing	process_pdfs(pdf_paths) → list[dict] canonical records
2. Retrieval	build_index(chunks); retrieve(question, top_k=5) → ranked original records
3. Generation & verification	generate_and_verify(question, passages) → structured result
4. Integration	answer_question(question, uploaded_docs) → final result for UI

3.4 Final response contract

answer: concise grounded answer or abstention. **citations:** document_id, document_name, page_number, chunk_ids and supporting_quote. **status:** one of the three defined statuses. **warnings:** unsupported claims, invalid IDs, page mismatches, OCR uncertainty, numerical mismatches or conflicts.

4.1 Non-functional requirements

ID	Requirement
NFR-01	Traceability: every displayed citation resolves to an indexed chunk and authoritative document/page metadata.
NFR-02	Grounding: generator uses supplied evidence only and abstains when support is insufficient.
NFR-03	Data integrity: extraction does not silently alter numbers, units, signs, decimals or table row relationships.
NFR-04	Reproducibility: record configuration, dataset/question IDs, model names and evaluation outputs.
NFR-05	Maintainability: separate processing, retrieval, generation, verification and UI into tested modules.
NFR-06	Portability: develop in Google Colab and keep reusable Python modules reasonably portable to local Python.
NFR-07	Security: never hard-code API keys; use environment variables/Colab Secrets and exclude secrets from Git.
NFR-08	Usability: clearly show answer, source document, page, evidence excerpt, status and warnings.
NFR-09	Error handling: report unreadable PDFs, extraction failures and missing model configuration explicitly.

4.2 Technology and setup

Area	Tools / packages
Environment	Python, Google Colab, private GitHub repository
PDF / OCR	pymupdf, pdfplumber, pytesseract, Tesseract system package, Pillow
Tables	pdfplumber; optional camelot-py
Retrieval	sentence-transformers, faiss-cpu; optional rank-bm25
Data / evaluation	pandas, numpy, scikit-learn, matplotlib
LLM / UI	Provider SDK behind configurable wrapper; streamlit; python-dotenv

Colab system setup: install tesseract-ocr and poppler-utils with apt. **Python setup:** install pymupdf, pdfplumber, pytesseract, pillow, pandas, numpy, sentence-transformers, faiss-cpu, streamlit, scikit-learn, matplotlib and python-dotenv. Install Camelot and rank-bm25 only if optional features are used.

4.3 Four-person ownership

Owner	Responsibility / deliverable
Member 1	PDF text extraction, OCR fallback, table extraction and canonical JSONL output.
Member 2	Chunking, embeddings, FAISS index, optional hybrid retrieval and retrieval tests.
Member 3	Grounded generation, citation validation, evidence support, numeric/conflict checks and status.
Member 4	Integration, Streamlit UI, baseline, evaluation metrics, plots, README and demo.

All members use the same schema, PDF collection, page-number convention and function signatures. Integrate a thin end-to-end slice early rather than waiting until all modules are finished.

5.1 Initial dataset and test set

The target dataset is 15 public factual PDFs: 5 text-based, 5 scanned and 5 containing tables or numerical information. Start integration with 3–5 representative PDFs, then expand to all 15. Prepare 50 manually checked questions: 10 direct factual, 10 multi-passage/multi-document, 10 table/numerical, 10 insufficient-evidence, 5 conflicting-source and 5 scanned/OCR-based. Each question includes an ID, question, reference answer or expected abstention, relevant document/page and category.

5.2 Acceptance criteria

- A user uploads multiple PDFs and asks a question across the collection.
- Digital text and scanned-page OCR are extracted with correct 1-indexed page provenance.
- Tables retain headers and rows; table answers identify the correct value, unit and period.
- Every retrieved chunk has a unique ID and authoritative document/page metadata.
- A multi-document question can cite more than one relevant source.
- An unsupported question returns Insufficient Evidence rather than a guessed answer.
- Invalid chunk IDs are rejected; displayed document names/pages come from metadata.
- Unsupported or conflicting claims produce warnings and an appropriate status.
- Baseline and TrustQA use the same fixed questions and documented configurations.
- Repository includes setup instructions, tests, evaluation outputs and a demo workflow.

5.3 Evaluation measures and comparison

Measure	Assessment
Answer quality	Exact Match / token F1 for short answers; manual or semantic rubric for longer answers.
Retrieval quality	Recall@K, Precision@K and MRR using labelled relevant chunks/pages.
Citation quality	Document/page provenance correctness and evidence-support correctness reported separately.
Trustworthiness	Unsupported-claim rate; correct abstention; detection of conflicting evidence.
Table / OCR quality	Numerical value, unit and period correctness; CER/WER where OCR ground truth exists.
Efficiency	End-to-end response latency; extraction/indexing time separately where useful.

Comparison: A) plain text RAG; B) OCR + plain RAG; C) TrustQA with OCR/text processing, table-aware representation, metadata-preserving retrieval and verification. Use the same question set and comparable model/retrieval settings. Do not invent scores; report measured results and failure examples.

5.4 Delivery roadmap and risks

Phase	Deliverable / exit condition
0. Freeze contracts	Agree schema, sample PDFs, question format and module signatures.
1. Baseline	Digital PDF → chunks → embeddings → retrieval → grounded answer with page source.
2. OCR + tables	Page-level OCR fallback and structure-preserving table extraction pass tests.
3. Verification	Citation registry, claim support, numeric checks, conflict and abstention pass tests.
4. Product demo	Streamlit upload/question/results interface works end to end.
5. Evaluation	Run 50 questions, compare baselines, save metrics, plots and error analysis.

Principal risks: OCR noise, irregular/scanned tables, unsupported LLM claims, false confidence from citation-only checks and integration delays. Mitigate through page-level OCR fallback, extraction inspection, preserved table structure, separate claim-support checks, shared schemas and early integration.

Final product outcome: a runnable prototype that returns not just an answer, but answer + evidence + source document + page + verification status, with reproducible evaluation against basic RAG.

Prepared from the TrustQA team research report and agreed implementation plan. The report's cited literature provides the research background; verify bibliographic details against original publications before formal submission.